



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza

Memoria de Práctica 1: Algoritmos Voraces (Hilorama)

Asignatura: Algoritmia Básica (Curso 2025/26)

Autores:

Imad Habib Jali

Jorge Bellido Lobera

901421@unizar.es

903080@unizar.es

1. Introducción y Decisiones de Diseño

El objetivo de esta práctica ha sido desarrollar un programa capaz de generar una representación artística en hiorama a partir de una imagen en escala de grises . Para ello, se ha implementado una heurística voraz (Greedy) que selecciona iterativamente las líneas (hilos) que mejor aproximan el lienzo de trabajo a la imagen original.

Decisiones de diseño principales:

- **Generación de candidatos:** En lugar de evaluar todas las posibles conexiones entre los n clavos (lo que supondría un coste inasumible de $n(n-1)/2$ combinaciones), se ha optado por generar aleatoriamente un subconjunto de tamaño p en cada iteración.
- **Trazado de líneas:** Para determinar los píxeles que componen cada hilo de forma eficiente y sin usar aritmética de coma flotante, se ha integrado el Algoritmo de Bresenham.
- **Función de evaluación (SAD):** La idoneidad de un hilo candidato se calcula mediante la Suma de Diferencias Absolutas. Un hilo obtiene mayor puntuación cuanto más coinciden sus píxeles con las zonas oscuras restantes en la imagen original. De los p candidatos evaluados, se seleccionan los s mejores ($s \ll p$).

2. Mejoras Implementadas

Se han añadido las siguientes mejoras para optimizar tanto la calidad visual como el rendimiento algorítmico:

2.1. Brillo, color y transparencia parametrizable (Apartado 2.5.1)

En lugar de asumir hilos perfectamente opacos y negros (valor 0) , se ha implementado un sistema que permite simular niveles de opacidad o intensidad del hilo. Este parámetro se puede ajustar en la llamada al programa. Su uso permite que el algoritmo saque partido de la superposición progresiva de hilos para generar gradientes y sombras mucho más realistas, evitando que una sola línea sature el lienzo de negro puro . Se ha fijado un valor por defecto para asegurar la compatibilidad con llamadas estándar.

2.2. Optimización por Evaluación Perezosa (Lazy Evaluation)

Al seleccionar los s mejores hilos de un lote de p candidatos, existe el riesgo de que varios hilos seleccionados crucen por la misma región oscura. Si se dibujan todos basándose en su puntuación inicial, la zona se sobresaturará (empeorando el error). Para solucionarlo, el programa recalcula la utilidad real del hilo en el momento exacto en el que va a ser dibujado. Si su puntuación ha caído por debajo de un umbral aceptable (debido a que hilos anteriores ya oscurecieron esa zona), se descarta.

3. Análisis de la Condición de Parada y Penalización de Calidad Visual

Durante el desarrollo se experimentó con la condición de parada basada en la estabilización del error global: detener el algoritmo cuando al añadir un nuevo hilo el error total aumenta o se mantiene igual.

Sin embargo, se observó una penalización en la calidad visual percibida al utilizar esta restricción estricta. Al tratarse de una heurística voraz, el algoritmo es propenso a caer en mínimos locales. Para trazar un contorno afilado o definir una sombra clave (rasgos fundamentales para el ojo humano), un hilo puede atravesar temporalmente zonas blancas, aumentando levemente el error matemático pixel a pixel. Si se detiene la ejecución ante este aumento, la imagen resultante queda indefinida y borrosa.

Por ello, se ha decidido utilizar un límite máximo de hilos (MAX_HILOS) y un umbral de utilidad mínima por hilo ($\text{scoreActual} < 10$) como condiciones de parada principales, priorizando el contraste humano frente a la optimización estricta del error matemático.

4. Experimentación y Análisis de Eficiencia

Se han realizado pruebas con diversas imágenes y parámetros para analizar el comportamiento espacial y temporal del algoritmo.

Análisis Teórico de Complejidad

Sea p el número de candidatos, s el número de seleccionados y L la longitud media de un hilo (proporcional al radio de la imagen).

En cada iteración:

1. Generar e inspeccionar candidatos tiene un coste temporal de $O(p \cdot L)$.
2. Ordenar los candidatos supone un coste de $O(p \log p)$.
3. El repintado final tiene un coste de $O(s \cdot L)$. Dado que $s \ll p$, el cuello de botella del algoritmo se encuentra en la exploración de los p candidatos. Reducir p disminuye drásticamente el tiempo de ejecución, pero degrada la calidad de la imagen resultante.

Resultados Empíricos

A continuación, se presentan los resultados obtenidos al procesar la imagen de prueba en la máquina *lab000*:

Imagen	N	p	s	Intensidad	Líneas Finales	Tiempo (s)	Error Final	% Mejora
p1	1000	2000	40	5	20280	3.11146s	15150850	71.0601%
p1	2500	5000	100	4	25400	4.50094s	14788669	71.7519%
p1	5000	10000	200	2	51000	9.09839s	14078243	73.1089%
p2	5000	10000	200	2	40000	5.28661s	29756786	39.8181%

Conclusiones de la experimentación:

Aumentar el valor de *p* incrementa el tiempo de ejecución linealmente, pero permite encontrar hilos de mayor calidad, reduciendo el error global final de forma más eficiente.

Al utilizar un valor alto de *intensidad* (hilos más oscuros), se alcanza el límite de contraste más rápido, requiriendo menos líneas en total pero generando un resultado con menor nivel de detalle en los gradientes suaves.